CHAPTERONE

Database-Driven Book Discovery and Reading Platform

DATABASE SYSTEMS LAB | PROJECT REPORT

Particular	Details
University / School	Vellore Institute of Technology - School of Computer Science and Engineering (SCOPE)
Course	BCSE302P - Database Systems Lab
Semester	Fall Semester 2026-27
Faculty	Dr. V. Sivakumar
Slot	L25+L26
Project Type	Solo / Individual Project
Student Name	[ENTER YOUR FULL NAME]
Registration Number	[ENTER YOUR REGISTRATION NUMBER]
Repository	https://github.com/CodeBloodMe/book-store

Prepared against the Lab Internal Assessment Test 8, 9 & 10 project rubric.

Document Guide

- 1. Executive Summary
- 2. Problem Statement and Project Planning
- 3. System Architecture and Scope
- 4. Database Design and ER Diagram
- 5. Relational Schema and Data Dictionary
- 6. Normalization, Constraints and Indexes
- 7. Database Implementation: CRUD, Joins, Aggregates, Views, Transactions, Function and Trigger
- 8. Application Development
- 9. Authentication, Authorization and Security
- 10. ORM Integration
- 11. Testing and Validation Plan
- 12. Backup, Recovery, Version Control, Docker, CI/CD and Deployment
- 13. Marking-Rubric Coverage Matrix
- 14. Final Submission Checklist
- Appendix A. Key SQL Demonstrations
- Appendix B. Evidence Still Required
- Appendix C. Full Relational Schema Notation
- Appendix D. SQL Source Code for Assessment Demonstrations

Evidence status convention: “Observed” means supported by repository code or supplied live-schema output; “Prepared” means a companion implementation file is included; “Pending” means it still requires execution or verification in the student’s environment. This distinction prevents the report from claiming tests or deployment that have not been performed.

1. Executive Summary

ChapterOne is a database-driven book discovery and reading platform. It maintains a catalog of books and genres, stores review and reading-list activity, and provides recommendation and book-relationship features on top of PostgreSQL. The web application is built with Next.js, React and TypeScript and connects to Supabase PostgreSQL. Additional components integrate Supabase Auth, external book metadata providers, vector similarity search and BookWeb graph traversal.

The DBMS assessment focuses on the relational data model and its implementation: entity relationships, primary and foreign keys, constraints, normalization, indexing, CRUD operations, joins, aggregates, views, transactions, a stored function, a trigger, ORM-backed interactions, role-based access, backups and deployment practice. This report maps those requirements to the current project and includes companion SQL scripts and integration templates.

The current source and schema evidence already show useful database features, including genre-to-book relationships, a self-reference for next-book navigation, review and shelf records, a weighted book graph, a ratings-summary view, a full-text search vector and a vector embedding column. However, having a feature in source code or a migration is not the same as proving it is deployed and tested. The remaining steps are explicit in the coverage matrix and submission checklist.

2. Problem Statement and Project Planning

2.1 Problem Statement

Readers often struggle to discover books that match their interests, preferred genres, reading difficulty, atmosphere or specific themes. A catalog that only supports exact title or author matching cannot fully support exploratory discovery, personalized reading lists, community feedback or analysis of relationships between books. ChapterOne addresses this need through a centralized relational database and a modern web interface that organize book metadata, genres, reviews, user shelves and graph relationships. It combines conventional database retrieval with optional AI-assisted search and recommendations.

2.2 Objectives

- Design a relational data model with at least eight related application tables and a documented ER diagram.
- Apply primary keys, foreign keys, uniqueness constraints, validation checks, defaults and appropriate indexes.
- Demonstrate CRUD, joins, aggregate queries, views, transactions, a stored function and a trigger.
- Integrate PostgreSQL with server-side application code through an ORM for applicable database operations.
- Provide authentication and authorization with a single role-bearing user profile table and at least two access levels.
- Demonstrate secure input handling, Row Level Security (RLS), secret management and exception handling.
- Validate database backup and recovery, version control, Docker configuration, automated checks and deployment.

2.3 Scope

In scope	Out of scope unless separately implemented
Book catalog, genre classification, book details and external metadata	Payments, online order fulfilment and physical inventory logistics
Reviews, user reading shelves and reader preference profiles	Commercial marketplace functions not present in the current application
BookWeb weighted edges and graph traversal	Claims that every graph relationship is factually verified without evidence
Database queries, views, stored functions, triggers, indexes and RLS	Claims of production availability or security certification without testing
Search, filtering, pagination, authentication and recommendation UI	Unverified operational metrics or fabricated test results

2.4 Solo project responsibilities

This is an individual project. The student is responsible for requirements analysis, schema design, ER modelling, SQL, backend/frontend implementation, ORM integration, authentication and authorization, testing, security review, backup/recovery, version control, deployment and documentation. Replace the placeholder student name and registration number on the cover before submission.

2.5 Technology stack

Layer	Technology / role	Status
Frontend	Next.js App Router, React, TypeScript, Tailwind CSS	Observed in repository
Backend/API	Next.js route handlers and server actions	Observed in repository

Database	PostgreSQL hosted by Supabase	Observed in repository and live-schema output
Authentication	Supabase Auth and cookie-based server client	Observed in repository
ORM	Prisma ORM	Schema file exists; real integration must be executed and evidenced
Search / AI	PostgreSQL full-text search, pgvector, Google GenAI and LLM provider fallback	Source-defined; runtime/configuration must be tested
Testing	Vitest and Playwright	Configuration and starter tests exist; run before claiming passing
Ops	GitHub, Docker, PostgreSQL backup/restore, Vercel	Configuration exists in part; operational evidence pending

2.6 Deliverables

- Project proposal and Gantt-style timeline.
- ER diagram, relational schema and data dictionary.
- Database SQL and migrations with constraints, indexes, view, function and trigger.
- CRUD/joins/aggregates/transactions demonstrations and timestamped result screenshots.
- ORM-backed endpoint, application pages and role-based access checks.
- Security audit, backup/restore test and operating instructions.
- Private GitHub repository with meaningful commit history, two branches and README.
- Docker/CI configuration and a deployed application URL/custom-domain evidence if completed.

2.7 Proposed timeline

Phase	Work packages	Evidence / exit criteria
1 - Planning	Problem statement, objectives, scope, stack, timeline	Approved/complete project proposal
2 - Database design	Schema audit, ERD, relational schema, constraints, normalization	Verified ERD and data dictionary
3 - DB implementation	CRUD, joins, aggregates, view, transactions, function, trigger, indexes	SQL results and constraint audit
4 - Application	Search, filter, pagination, UI, validation, exceptions, ORM	Working routes and screenshots
5 - Security	Auth profile, roles, RLS, API authorization and secrets	Positive/negative authorization tests
6 - Professional practice	Backup/restore, Git branches/commits, Docker and CI	Successful run logs and separate DB restore test
7 - Deployment	Deploy, smoke test, custom domain if configured	Live URL and timestamped evidence

Timeline note: this is a proposed phase plan, not a record of work dates. Replace it with actual dates/progress before submission.

3. System Architecture and Scope

The browser renders the Next.js interface. Server components, route handlers and server actions communicate with Supabase PostgreSQL through the Supabase client; selected operations are intended to use Prisma after the included integration steps are completed. Supabase Auth handles identity and sessions. PostgreSQL holds normalized catalog entities and database-side features. External book providers can enrich metadata, while AI and embeddings support semantic retrieval and generated summaries.

Layer	Responsibilities
Browser / Next.js UI	Renders pages and captures user input.
Next.js server components, route handlers and server actions	Validates requests and mediates all server-side data access.
Supabase Auth	Authenticates identity and maintains the session.
PostgreSQL / Supabase	Stores books, genres, reviews, shelves, graph edges and database-side logic.
External book APIs / AI providers	Optional metadata enrichment, embeddings and recommendation generation.

Architecture note: the primary catalog and account state are persistent relational data. AI output and graph path selection are not substitutes for database constraints or evidence of relationship correctness.

4. Database Design and ER Diagram

The current-state ER diagram is generated from the schema evidence supplied by the student and the repository migrations. It shows the catalog hierarchy, self-referencing next-book link, reviews, shelves, graph edges and reader interactions. The diagram is intentionally honest about schema gaps: the author name in books is stored as text and is not formally connected to authors.id; blind_date_interactions.session_id is text and lacks an FK to user_reader_profiles.session_id in the supplied migration; Supabase auth.users is a provider-managed external relation.

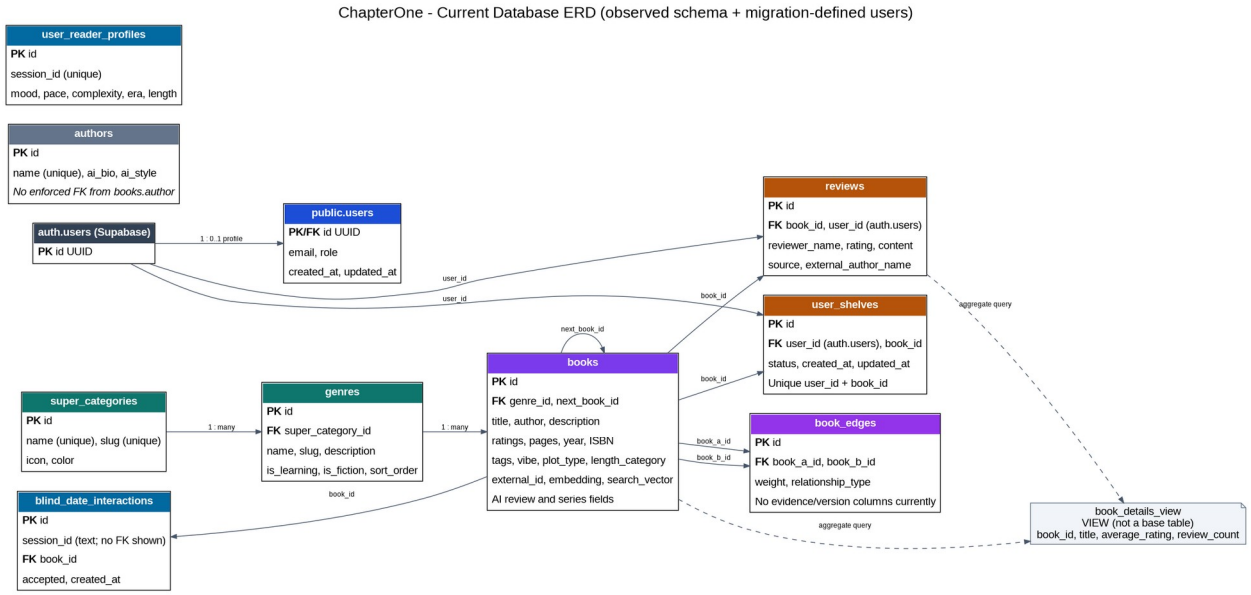


Figure 1. ChapterOne current-state database ERD (app tables plus Supabase Auth reference).

The live column export confirmed core fields for authors, books, book_edges, blind_date_interactions, reviews, genres, super_categories, user_reader_profiles, user_shelves and the book_details_view. The export was truncated during user_shelves, so the complete live table/constraint listing is still required. The public.users table is defined in the repository migration but should be confirmed in the live audit before the diagram is treated as final.

4.1 Main relationships

Relationship	Cardinality	Implementation / caveat
Super-category to genre	1 : many	genres.super_category_id references super_categories.id
Genre to book	1 : many	books.genre_id references genres.id
Book to next book	0/1 : many incoming links	books.next_book_id is a self-reference
Book to review	1 : many	reviews.book_id references books.id
Auth user to review	1 : many / optional user for review	reviews.user_id references auth.users.id in migration
Auth user to shelf rows	1 : many	user_shelves.user_id references auth.users.id
Book to shelf rows	1 : many	user_shelves.book_id references books.id; unique(user_id, book_id)
Book to graph edge	1 : many at each endpoint	book_edges.book_a_id and book_b_id both reference books.id
Reader profile to interaction	Conceptual by session_id	No FK is defined in provided migration; referential integrity can be improved after audit
Author to book	Not enforced	books.author is a text field, not an FK to authors.id

4.2 Minimum table count

The repository defines at least nine application-owned base tables visible in the reviewed schema/migrations: super_categories, genres, books, authors, reviews, user_shelves, book_edges, user_reader_profiles, and blind_date_interactions. The migration-defined public.users table is an additional candidate and must be verified in the full live audit. book_details_view is a view and does not count as a base table. auth.users is managed by Supabase and should not be counted as an application-owned table.

5. Relational Schema and Data Dictionary

The following schema summary uses SQL object names and key notation. It is a data dictionary view of the known implementation; all constraint names and deployed relationships should be reconciled against the output of sql/00_schema_audit.sql.

Relation	Key fields / purpose	Integrity notes
super_categories	id PK; name, slug, icon, color	name and slug unique in seed schema
genres	id PK; super_category_id FK; name, slug, flags, sort_order	super_category_id NOT NULL; name and slug unique in seed schema
books	id PK; genre_id FK; next_book_id self-FK; title, author, details, ratings, tags, embedding	external_id unique; ratings expected 0..5; difficulty/length have CHECKs in seed schema
authors	id PK; name, ai_bio, ai_style, timestamps	name UNIQUE in create_authors_table.sql; ai_style array is multi-valued
reviews	id PK; book_id FK; optional user_id; reviewer_name, rating, content, source	rating expected 1..5; historical policies vary across migrations
user_shelves	id PK; user_id FK, book_id FK, status, timestamps	unique(user_id, book_id); status constrained to three reading states
book_edges	id PK; book_a_id FK, book_b_id FK, weight, relationship_type	unique ordered pair in migration; check book_a_id <> book_b_id; this alone does not prevent reverse duplicate pairs
user_reader_profiles	id PK; session_id, mood, pace/complexity/era/length preferences	session_id UNIQUE in migration
blind_date_interactions	id PK; session_id, book_id FK, accepted, created_at	session_id is text, no FK to profile in the supplied migration
users	id PK/FK to auth.users; email, role, timestamps	defined in 9999_dbms_requirements.sql; verify live; role check uses one profile table
book_details_view (VIEW)	book_id, title, average_rating, review_count	derived from books and reviews; not a base table

5.1 Other important book columns

The observed books table also includes ISBN and publication year; cover and marketplace URLs; expert quote/name; difficulty level; tags, vibe, plot_type and length_category; curation flags; created_at/updated_at; search_vector; series_name/series_number; external_id; AI review summary/pros/cons/rating/timestamp; fiction_genres; free_reading_url; embedding; author_image_url; and author_bio. PostgreSQL-specific fields such as tsvector and vector require native-type awareness during ORM introspection.

6. Normalization, Constraints and Indexes

6.1 Normalization assessment

Normal form	Principle	ChapterOne assessment / action
1NF	Each field stores a single value under the classical relational model.	Array columns tags, fiction_genres, ai_style, ai_pros and ai_cons are multi-valued. PostgreSQL arrays are valid features, but a strict normalized relational design may model them as child/bridge tables. Document the chosen approach and its

		trade-off.
2NF	Every non-key attribute depends on the whole candidate key.	Most current main tables use a single-column UUID primary key. For proposed bridge tables such as book_authors/book_tags, use composite keys and avoid attributes dependent on only one part.
3NF	Non-key attributes should not depend transitively on a key.	Author biography is separately stored in authors while books also has author_bio/author_image_url; this may duplicate facts. genre.book_count and books.total_reviews are counters that can drift unless maintained. Derive counts in views/queries or define a documented maintenance rule.

Do not claim “fully normalized to 3NF” without explaining arrays, cached counts and the duplicated author-related fields. For the practical assessment, describe the current schema accurately, state the normalization limitations, and implement only changes that have been migrated and tested.

6.2 Key constraints

- Primary keys use UUIDs and default UUID generators where configured.
- Genre and super-category slugs/names are intended to be unique.
- Book external_id is intended to be unique so provider imports can be deduplicated.
- Book ratings are intended to be within 0..5; review ratings are within 1..5.
- User shelves enforce one row per user/book pair and valid reading statuses.
- Book edges prohibit self-loops and duplicate stored orientation, but reverse-edge duplicates require a canonical-pair constraint or carefully controlled graph builder.
- Foreign keys define behavior on deletion, including cascade and set-null cases; verify the actual deployed rules with the schema audit.

6.3 Indexes

The seed script defines a GIN full-text-search index for books.search_vector, indexes on books.genre_id, (genre_id, difficulty_level), expert_rating, next_book_id, a partial index on is_featured, and indexes on genres.slug and genres.super_category_id. Other migrations define indexes for reviews and book_edges. The pgvector migration defines an HNSW cosine-distance index on books.embedding. Keep indexes that support actual query plans; avoid duplicating the same index and inspect EXPLAIN ANALYZE before adding more.

7. Database Implementation

7.1 CRUD

sql/02_assessment_queries.sql contains a repeatable demonstration record identified by external_id = dbms-demo:chapterone-001. It illustrates insert/upsert, read, update and a commented delete that is restricted to the dedicated test record. The record should be created in a controlled test session and removed after capturing evidence if desired. Do not run broad deletes or rerun the destructive seed script against a live database.

```
-- Read example
SELECT id, title, author, page_count
FROM public.books
WHERE external_id = 'dbms-demo:chapterone-001';
```

7.2 Joins and aggregate queries

The book/genre/super-category join demonstrates normalized lookup across the catalog hierarchy. A LEFT JOIN between books and reviews preserves books that have no reviews; AVG and COUNT then summarize the feedback. Grouping by genre gives book counts for category analytics. These are meaningful relational operations, not just isolated SELECT statements.

```
SELECT b.id, b.title, g.name AS genre, sc.name AS super_category
FROM public.books b
JOIN public.genres g ON g.id = b.genre_id
JOIN public.super_categories sc ON sc.id = g.super_category_id;
```

7.3 View

The repository migration 9999_dbms_requirements.sql defines public.book_details_view, returning each book title, average review rating and review count. It avoids storing another independently maintained aggregate. The existence of this view is confirmed by the live column export; its definition and grants should still be checked in the audit.

7.4 Transaction

The transaction example updates only the dedicated demo record, reads the uncommitted value in the same transaction, and then issues ROLLBACK. This demonstrates atomicity and recovery from a transaction without leaving a persistent change. Capture both the value before and after the rollback to show that the update was undone.

```
BEGIN;
UPDATE public.books SET page_count = COALESCE(page_count, 0) + 1
WHERE external_id = 'dbms-demo:chapterone-001';
SELECT page_count FROM public.books WHERE external_id = 'dbms-demo:chapterone-001';
ROLLBACK;
```

7.5 Stored function and trigger

sql/03_book_rating_function.sql provides a guarded stored-function demonstration that validates a numeric rating and rejects a missing book. It deliberately restricts execution to trusted database roles; do not grant this mutation function to anonymous clients as written. sql/04_books_updated_at_trigger.sql provides a trigger that maintains books.updated_at on updates. The existing seed schema also defines a trigger for refreshing books.search_vector. Verify actual live trigger definitions before claiming all triggers are active.

7.6 Performance evidence

Run the EXPLAIN (ANALYZE, BUFFERS) example against a representative catalog query and save the plan. Index selection should be supported by observed execution plans rather than a generic claim that adding indexes always improves performance. On small tables, a sequential scan can be optimal.

8. Application Development

The repository includes responsive Next.js pages and feature components for the homepage, genre browsing, search, individual book details, recommendations, BookWeb, Blind Date, login, saved books and the EPUB reader. The application already has route-level error handling in several endpoints and test scaffolding for homepage/search behavior.

Requirement	Existing evidence / implementation area	Remaining verification
Responsive UI	Next.js pages, Tailwind styles, reusable components	Capture desktop and mobile screenshots

Login/logout	app/login and Supabase server client	Test valid, invalid and expired-session cases
Dashboard / library	app/my-books and user_shelves	Test shelf ownership and empty/error state
CRUD	shelves, reviews and database queries	Document record creation/update/deletion with DB results
Search/filter	app/search, lib/queries.ts and book search APIs	Test exact, partial, empty and special-character queries
Pagination	ORM sample route included in companion files	Integrate and capture page navigation results
Validation	review and request validation exists in parts	Add boundary tests and validation for all public APIs
Exception handling	many routes return JSON errors	Test provider outage, DB error and invalid identifiers

9. Authentication, Authorization and Security

9.1 Authentication

Supabase Auth handles user identity. The project uses login/signup server actions and a cookie-aware server client. Authentication establishes who the user is; it does not automatically establish what actions the user is allowed to perform.

9.2 Role-based authorization

The repository migration defines one public.users profile table with a role column and a check over Admin, User, Faculty and Student. The application must retrieve the authenticated user's server-side profile and enforce role checks on protected pages and APIs. The included optional sql/06_auth_rbac_setup.sql creates/backfills a default User profile and a signup trigger without trusting user-supplied metadata to choose a role. The supplied ORM integration contains a requireRole helper and Admin-only count endpoint. Integrate and test these files before claiming RBAC is complete.

A role field in a database is not enough. A user must not be able to update their own role to Admin. Role changes should be made only through a trusted administrative route or database-owner process. Every API route that mutates protected data must enforce authorization server-side; hiding a UI button is insufficient.

9.3 RLS and API security

- Enable and inspect RLS on user-owned records; user shelves should be readable/writable only when auth.uid() equals user_id.
- Keep public book catalog reads separate from user-specific writes.
- Review review policies so a user cannot delete or update somebody else's review.
- Public reader-session interaction writes require validation and rate limiting; never accept any book/session combination without checks.
- Book graph mutation and bulk catalog population should require a trusted service-side role.
- Keep SUPABASE_SERVICE_ROLE_KEY, DATABASE_URL and private AI keys out of browser bundles and Git history.
- Move hardcoded configuration out of scripts into environment variables; remove any committed real credentials and rotate secrets if a secret was exposed.
- Avoid API secrets in URLs because URLs are commonly logged. Use authorization headers and fail closed if production secrets are missing.
- Use parameterized ORM/query APIs for user inputs; never concatenate raw user strings into SQL.

The security audit script is read-only. It reports policies, grants and SECURITY DEFINER functions for review; it does not automatically rewrite every policy because doing so without understanding app behavior can break login, public catalog access or anonymous Blind Date sessions.

10. ORM Integration

The existing repository contains prisma/schema.prisma, but the principal database-access modules inspected use Supabase JS. A schema file by itself does not establish that Prisma is installed, generated or used in application execution. The included orm-integration directory supplies a server-only Prisma client singleton, a paginated/searchable GET route, and a role-check example. The setup instructions require adding Prisma dependencies, setting a server-only DATABASE_URL, introspecting the real schema and generating the client.

Use Prisma for applicable relational read/write paths where it is appropriate; keep Supabase Auth and session-aware RLS access via the Supabase server client. Never run Prisma schema-push/migrations against the live database without review and backup. Capture an actual ORM route response to demonstrate execution.

11. Testing and Validation Plan

Test category	Test cases	Evidence to capture
Schema	Tables, views, PK/FK/unique/check constraints, indexes, triggers, RLS	Saved results from sql/00_schema_audit.sql
Integrity	Invalid ratings, self-loops, duplicate undirected pairs, orphan session references	Saved results from sql/01_data_integrity_audit.sql
CRUD	Demo record insert, retrieve, update and optional cleanup	SQL editor results with timestamp
Joins/aggregates	Genre/category join, average review ratings, review-source counts	Result-grid screenshots with query visible
View/function/trigger	Read book_details_view; invalid/valid function inputs; updated_at trigger	Before/after query output
Transactions	Update demo row inside transaction then rollback	Value before, during and after rollback
Authentication/RBAC	Anonymous, regular user, admin; attempt forbidden role escalation	HTTP status/body screenshots and server logs without secrets
Application	Search, filtering, pagination, form validation, responsive layout	Playwright results and timestamped screenshots
Recovery	pg_dump and restore into separate test DB; row counts/smoke queries	Backup file info, restore command result and validation queries
CI/build	lint, unit tests, type-check, production build	GitHub Actions green run or terminal logs

Existing tests cover external book API parsing and simple homepage/search browser behavior. They do not by themselves validate graph optimality, live RLS, transaction rollback, ORM integration, backup recovery or RBAC. Run the tests in the target environment and only report outcomes actually observed.

12. Backup, Recovery, Version Control, Docker, CI/CD and Deployment

12.1 Backup and recovery

PostgreSQL native tools pg_dump and pg_restore are the expected mechanisms. The included ops/backup_restore.md demonstrates a custom archive backup and a restore into a separate test database.

Keep dump files out of Git, protect them as potentially sensitive data, and validate the restored result rather than treating a successful dump command as proof of recoverability.

12.2 GitHub workflow

- Use a private repository for the assessment and add any required collaborators.
- Maintain main and dev branches and commit meaningful changes incrementally.
- Keep at least ten meaningful commits; do not manufacture history or commit unrelated filler changes.
- Keep .env, .env.local, backup files, database dumps and private keys out of version control.
- Use a README containing description, stack, setup, environment variables and run instructions.

The companion ci/main.yml is a GitHub Actions template. Copy it to .github/workflows/main.yml in the project and configure the required GitHub Secrets. Run lint, tests, type checking and production build; a YAML file alone is not proof that CI/CD passes.

12.3 Docker and deployment

A Dockerfile exists in the current repository. Add the supplied .dockerignore, build the image locally and start it with valid server-side environment variables. Deploy the application to the chosen host, configure Supabase URLs/auth redirect URLs as needed, smoke-test key routes and record the live URL. A custom domain is only complete after it is configured at the hosting provider and verified in a browser.

12.4 Additional best practice

Automated testing, structured error logging, database performance measurement and backup recovery validation are appropriate options for the additional-industry-practice mark. The most defensible choice is automated testing plus a tested restore procedure, with dated outputs saved as evidence.

13. Marking-Rubric Coverage Matrix

Rubric area	Marks	Prepared / observed evidence	Final evidence status
Problem statement & planning	2	Sections 2.1-2.7; student identity placeholders	Complete after fields/date verification
Database design	4	Current ERD, relational schema, normalization and constraints review	Needs full live constraint audit; review normalization choices
Database implementation	7	SQL audit, CRUD, joins, aggregates, view, transaction, function and trigger scripts	Run and capture result evidence
Application development	4	Existing Next.js pages/routes plus ORM pagination sample	Run UI tests; integrate pagination/ORM proof
Auth, RBAC & security	5	Supabase Auth observed; optional safe profile/RBAC setup and route example	Apply after backup; test positive/negative cases
Professional development	5	Dockerfile and starter tests exist; backup/recovery and Git checklist provided	Private repo, commits, two branches, restore and image evidence pending
CI/CD	1	Workflow template included	Install/configure and obtain successful run
Custom domain	1	Deployment checklist	Configure and verify externally
Additional best practice	1	Automated checks and restore validation plan	Provide successful execution evidence

This matrix reflects readiness, not marks achieved. No score is guaranteed; final evaluation depends on the instructor's checks and evidence.

14. Final Submission Checklist

☐ Fill student name, registration number and exact slot in the report and screenshots.

☐ Run the full read-only schema audit and reconcile the ERD with deployed constraints and tables.

☐ Run data integrity checks; document and fix real findings safely.

☐ Create/verify at least eight to ten related application tables with all represented in the ERD.

☐ Run CRUD, join, aggregate, view and transaction demonstrations and save timestamped screenshots.

☐ Install/generate Prisma, use the ORM route, and capture a real response.

☐ Create/verify stored function and trigger in the live database, then test valid and invalid inputs.

☐ Verify login/logout, user profile, two roles, role-restricted page/API, RLS and forbidden access cases.

☐ Use private GitHub repository, main/dev branches, collaborator settings, ten meaningful commits and complete README.

☐ Run automated tests, lint, TypeScript check and production build; retain logs.

☐ Run pg_dump and restore into a separate test database; validate row counts and query functionality.

☐ Build Docker image and smoke-test the container.

☐ Configure live hosting and custom domain if required; capture URL and screenshots.

☐ Replace every pending claim with observed evidence; do not submit fake outputs or unverified metrics.

☐ Prepare PDF filename using the faculty's exact convention: REGISTERNO_SLOT_IA9&10_Proj_NAME.pdf; upload PDF to VTOP and relevant PDF/DOC to Moodle as instructed.

Appendix A. Key SQL Demonstrations

A1. Average review rating per book

```
SELECT b.id, b.title,
    ROUND(COALESCE(AVG(r.rating), 0)::numeric, 2) AS average_rating,
    COUNT(r.id) AS review_count
FROM public.books b
LEFT JOIN public.reviews r ON r.book_id = b.id
GROUP BY b.id, b.title
ORDER BY review_count DESC, b.title
LIMIT 30;
```

A2. Book and genre hierarchy

```
SELECT b.title, g.name AS genre, sc.name AS super_category
FROM public.books b
JOIN public.genres g ON g.id = b.genre_id
JOIN public.super_categories sc ON sc.id = g.super_category_id
ORDER BY sc.name, g.name, b.title;
```

A3. View-based analytics

```
SELECT book_id, title, average_rating, review_count
FROM public.book_details_view
ORDER BY average_rating DESC, review_count DESC;
```

Appendix B. Evidence Still Required

Evidence item	What counts as proof	Status
Live schema/keys	Output from 00_schema_audit.sql	PENDING
CRUD + SQL queries	Timestamped SQL-editor screenshots/results	PENDING
View/function/trigger	Catalog definitions and successful executions	PENDING
ORM	Generated Prisma client and GET /api/dbms/books response	PENDING
RBAC/RLS	Successful and rejected requests for each role	PENDING
Tests/build	Actual local or CI logs	PENDING
Backup/restore	Separate database restore and validation results	PENDING
GitHub	Private repo, branches, meaningful history, .gitignore	PENDING
Docker/deployment/domain	Successful image run, live URL and custom-domain check	PENDING

Final integrity note: this report documents observed source/schema information and a set of prepared implementation scripts. It intentionally does not claim that pending SQL, Prisma integration, tests, backup recovery, CI, deployment or custom-domain tasks have already succeeded. Verify each task and capture evidence before submission.

Appendix C. Full Relational Schema Notation

The complete column-level notation is also provided in sql/09_relational_schema.md within the companion package. The following compact schema identifies each base relation and its key relationships. Constraints and data types should be finalized using the live-schema audit.

```
SUPER_CATEGORIES(id PK, name UK, slug UK, icon, color)
GENRES(id PK, super_category_id FK -> SUPER_CATEGORIES.id, name UK, slug UK,
    icon, description, color, is_learning, is_fiction, book_count, sort_order)
BOOKS(id PK, genre_id FK -> GENRES.id, next_book_id FK -> BOOKS.id,
    title, author, description, cover_image_url, page_count, published_year,
    isbn, language, amazon_url, goodreads_url, expert_rating, community_rating,
    total_reviews, expert_quote, expert_name, difficulty_level, tags, vibe,
    plot_type, length_category, is_featured, is_editors_pick, is_bestseller,
    created_at, updated_at, search_vector, series_name, series_number,
    external_id UK, ai_review_summary, ai_pros, ai_cons, ai_rating,
    ai_last_updated, fiction_genres, free_reading_url, embedding,
    author_image_url, author_bio)
AUTHORS(id PK, name UK, ai_bio, ai_style, ai_last_updated, created_at, updated_at)
REVIEWS(id PK, book_id FK -> BOOKS.id, reviewer_name, rating, content,
    created_at, user_id FK -> auth.users.id NULL, source, external_author_name)
USER_SHELVES(id PK, user_id FK -> auth.users.id, book_id FK -> BOOKS.id,
    status, created_at, updated_at, UK(user_id, book_id))
BOOK_EDGES(id PK, book_a_id FK -> BOOKS.id, book_b_id FK -> BOOKS.id,
    weight, relationship_type, created_at, UK(book_a_id, book_b_id),
    CHECK(book_a_id <> book_b_id))
USER_READER_PROFILES(id PK, session_id UK, mood, pace_preference,
    complexity_preference, era_preference, length_preference, created_at)
BLIND_DATE_INTERACTIONS(id PK, session_id, book_id FK -> BOOKS.id,
    accepted, created_at)
USERS(id PK/FK -> auth.users.id, email, role, created_at, updated_at)
BOOK_DETAILS_VIEW(book_id, title, average_rating, review_count) -- VIEW, not a base table
```

Notable schema limitations: books.author is not a foreign key to authors.id; array attributes include tags, fiction_genres and ai_style; session_id on blind_date_interactions has no FK to user_reader_profiles in the supplied migration; book_edges uniqueness is directional unless canonical endpoints are enforced. These are documented design limitations rather than hidden inferences.

Appendix D. SQL Source Code for Assessment Demonstrations

This appendix includes the complete primary assessment query, stored-function and trigger scripts in text form. The companion package contains the separate schema-audit, integrity-audit, security-audit, optional RBAC and fresh-database reference SQL files. Execute scripts against the intended database only after reading their comments and backup requirements.

D1. CRUD, JOIN, Aggregate, View, Search, Pagination and Transaction Queries

```
-- ChapterOne DBMS lab query demonstrations
-- Read the comments before running. The demo book uses a dedicated external_id.


-- A. CREATE (repeatable demo record). Uses the first available genre so no genre slug is assumed.
INSERT INTO public.books
    (genre_id, title, author, description, language, page_count,
     expert_rating, community_rating, total_reviews, external_id,
     is_featured, is_editors_pick, is_bestseller, tags)
SELECT g.id, 'ChapterOne DBMS Demonstration Book', 'ChapterOne Demo Author',
       'Temporary record used to demonstrate database operations.', 'English',
       120, 4.0, 4.0, 0, 'dbms-demo:chapterone-001',
       false, false, false, ARRAY['DBMS_DEMO']::text[]
FROM public.genres g
ORDER BY g.sort_order NULLS LAST, g.name
LIMIT 1
ON CONFLICT (external_id) DO UPDATE
SET description = EXCLUDED.description,
    updated_at = now()
RETURNING id, title, external_id;

-- B. READ: retrieve a record by its stable demo identifier
SELECT id, title, author, page_count, expert_rating, external_id
FROM public.books
WHERE external_id = 'dbms-demo:chapterone-001';

-- C. UPDATE: change only the demo row
UPDATE public.books
SET description = 'Updated description for the DBMS CRUD demonstration.',
    page_count = 125,
    updated_at = now()
WHERE external_id = 'dbms-demo:chapterone-001'
RETURNING id, title, description, page_count;

-- D. DELETE: run only after all demonstrations are finished; affects only the dedicated demo record.
-- DELETE FROM public.books WHERE external_id = 'dbms-demo:chapterone-001';

-- E. JOIN: book -> genre -> super-category
SELECT b.id, b.title, b.author, g.name AS genre, sc.name AS super_category
FROM public.books b
JOIN public.genres g ON g.id = b.genre_id
JOIN public.super_categories sc ON sc.id = g.super_category_id
ORDER BY sc.name, g.name, b.title
LIMIT 30;

-- F. LEFT JOIN + aggregate: review statistics per book (books with zero reviews remain visible)
SELECT b.id, b.title,
    ROUND(COALESCE(AVG(r.rating), 0)::numeric, 2) AS average_review_rating,
    COUNT(r.id) AS review_count
```

```
FROM public.books b
LEFT JOIN public.reviews r ON r.book_id = b.id
GROUP BY b.id, b.title
ORDER BY review_count DESC, b.title
LIMIT 30;

-- G. Aggregate: number of books per genre
SELECT g.name AS genre, COUNT(b.id) AS book_count
FROM public.genres g
LEFT JOIN public.books b ON b.genre_id = g.id
GROUP BY g.id, g.name
ORDER BY book_count DESC, g.name;

-- H. Aggregate: review count and average rating by source
SELECT COALESCE(source, 'unknown') AS review_source,
      COUNT(*) AS total_reviews,
      ROUND(AVG(rating)::numeric, 2) AS average_rating
FROM public.reviews
GROUP BY COALESCE(source, 'unknown')
ORDER BY total_reviews DESC;

-- I. VIEW: view object is created by the existing 9999_dbms_requirements.sql migration
SELECT book_id, title, average_rating, review_count
FROM public.book_details_view
ORDER BY average_rating DESC, review_count DESC
LIMIT 30;

-- J. Filtering and sorting
SELECT id, title, author, page_count, published_year
FROM public.books
WHERE page_count BETWEEN 100 AND 400
  AND (title ILIKE '%python%' OR author ILIKE '%python%')
ORDER BY title ASC
LIMIT 20;

-- K. Pagination: page 2 when each page contains 10 rows
SELECT id, title, author
FROM public.books
ORDER BY title, id
LIMIT 10 OFFSET 10;

-- L. Index-assisted plan inspection; compare the plan before/after any index changes.
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, title FROM public.books WHERE genre_id = (SELECT id FROM public.genres ORDER BY name LIMIT 1);

-- M. Transaction demonstration (no lasting change). Run after the demo INSERT above.
BEGIN;
UPDATE public.books
SET page_count = COALESCE(page_count, 0) + 1, updated_at = now()
WHERE external_id = 'dbms-demo:chapterone-001';
SELECT id, title, page_count FROM public.books WHERE external_id = 'dbms-demo:chapterone-001';
ROLLBACK;
-- After ROLLBACK, page_count returns to its pre-transaction value.
```

D2. Stored Function

```
-- ChapterOne stored-function demonstration.
```

```
-- SECURITY INVOKER (default), fixed search_path, rejects invalid values and missing book IDs.
-- Execution permissions are restricted here to trusted database roles; verify your role names first.
CREATE OR REPLACE FUNCTION public.set_book_expert_rating(p_book_id uuid, p_rating numeric)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = pg_catalog, public
AS $$
BEGIN
  IF p_rating IS NULL OR p_rating < 0 OR p_rating > 5 THEN
    RAISE EXCEPTION 'expert rating must be between 0 and 5';
  END IF;

  UPDATE public.books
    SET expert_rating = p_rating,
      updated_at = now()
   WHERE id = p_book_id;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'book % was not found', p_book_id;
  END IF;
END;
$$;

-- Avoid exposing a rating mutation RPC to anonymous clients.
REVOKE ALL ON FUNCTION public.set_book_expert_rating(uuid, numeric) FROM PUBLIC;
REVOKE ALL ON FUNCTION public.set_book_expert_rating(uuid, numeric) FROM anon;
REVOKE ALL ON FUNCTION public.set_book_expert_rating(uuid, numeric) FROM authenticated;

-- Demonstration in the SQL editor (choose a real ID; do not use arbitrary input):
-- SELECT public.set_book_expert_rating('<existing-book-uuid>'::uuid, 4.25);
-- This function should be called only by a trusted database owner/administrative process unless
-- a reviewed authorization design is added. Do not grant it to public app users as written.
```

D3. Trigger

```
-- ChapterOne trigger demonstration: keep books.updated_at current on UPDATE.
-- This script is idempotent for the trigger name below and does not change existing rows.
CREATE OR REPLACE FUNCTION public.chapterone_set_books_updated_at()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = pg_catalog, public
AS $$
BEGIN
  NEW.updated_at := now();
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS chapterone_books_updated_at ON public.books;
CREATE TRIGGER chapterone_books_updated_at
BEFORE UPDATE ON public.books
FOR EACH ROW
EXECUTE FUNCTION public.chapterone_set_books_updated_at();

-- Verify the trigger from the schema audit query, then update the dedicated demo row to test it.
-- UPDATE public.books SET title = title WHERE external_id = 'dbms-demo:chapterone-001';
```